AWS Copilot

Explore how to deploy and manage containerized applications using AWS Copilot, focusing on ECS and Fargate. Understand Copilot's components, environment isolation, CI/CD integration, IAM roles, secure networking, monitoring, and deployment resilience to streamline cloud-native microservices development and operations.

AWS Copilot is an opinionated CLI tool that helps developers deploy containerized applications to AWS App Runner, Amazon ECS, and AWS Fargate with minimal configuration. In this lesson, we'll focus on how Copilot works with ECS, as AWS App Runner is out of scope for this exam.

Copilot eliminates the need to manually set up infrastructure by generating and managing AWS CloudFormation templates behind the scenes. This significantly simplifies the process of defining and operating cloud-native microservices.

Designed to support Amazon ECS on both Fargate and EC2, Copilot is especially effective when paired with Fargate's serverless compute model, which abstracts away server management tasks. While ECS on EC2 is still supported for advanced customization, most users will benefit from Fargate's streamline deployment experience.

Ask

From a developer's perspective, AWS Copilot offers several advantages:

- It creates consistent, production-ready infrastructure using proven AWS architectural patterns.
- It accelerates deployment with high-level commands that encapsulate best practices.
- It reduces manual configuration by handling IAM, networking, logging, monitoring, and deployment strategy setup automatically.

Copilot allows us to focus on writing code, not infrastructure. It supports real-world workflows such as environment separation (e.g., dev, test, prod), CI/CD integration, and safe deployments with rollback. The illustration below shows how deployments are managed using Copilot.

Deploying ECS resources using Copilot

Ask

Before exploring the application lifecycle, it's helpful to understand the main components of AWS Copilot.

Core components of AWS Copilot

The main components that Copilot uses to structure and manage applications are as follows:

- **Application:** It is a top-level logical construct that represents the entire deployment context for the project. It groups related services and jobs under a single namespace and allows shared configuration across environments. Each application maps to an AWS CloudFormation StackSet and manages metadata, enabling centralized resource tracking and organizational consistency.
- **Manifest:** It is a YAML file that describes the configuration of a service or job. It defines container port mappings, CPU/memory limits, environment variables, auto scaling policies, secrets, and health checks. Manifests are version-controlled, environment-aware, and drive the behavior of Copilot deployments.
- **Service:** It is a containerized workload that is continuously running, often representing user-facing functionality such as an API or web frontend. Services are automatically deployed behind load balancers when needed, configured with auto scaling, and integrated with logging and health checks. Each service has its own manifest file to specify runtime behavior.
- **Job:** It is a task that runs to completion and then exits. Jobs are suitable for batch processing, scheduled data workflows, or one-off initialization scripts. Jobs can be scheduled using cron expressions and can interact with other AWS services like S3, DynamoDB, or EventBridge.
- **Environment:** It is a self-contained deployment target that includes it networking stack (VPC, subnets, and NAT gateways), IAM roles, and service discovery namespace. Copilot environments are explicitly defined, like `dev`,

Ask

`test`, `prod`, and allow for isolated deployments with environment-specific configuration and security boundaries.

- **Pipeline:** It is a CodePipeline configuration automatically generated and deployed by Copilot to support CI/CD. The pipeline integrates with Git repositories, builds container images with CodeBuild, and deploys them through one or more Copilot environments. It supports manual approvals, stage promotions, and artifact reuse for secure and consistent deployments.

> AWS CodePipeline is not supported for services with Windows as the OS Family. CodePipeline uses Linux-based AWS CodeBuild for the build stage, so for now, Copilot pipelines cannot build Windows containers.

Each component is created, managed, and connected using Copilot CLI commands. Together, they form the structural and operational foundation of a Copilot-based application.

Application lifecycle with Copilot

AWS Copilot manages the end-to-end lifecycle of containerized applications. It organizes the workloads into services or jobs, provisions isolated environments, and handles deployment and monitoring. This lifecycle approach allows us to iteratively build and release features, using a consistent workflow across different stages of development.

Each step, from service initialization to environment configuration and multistage deployment, is integrated into a single toolchain. This reduces operational overhead and gives developers full control and visibility without needing to manage individual AWS components manually. Copilot's structure also supports scalability and maintainability by enforcing conventions around environment separation, deployment strategy, and configuration management.

Ask

Let's examine a sample deployment using AWS Copilot and understand how to deploy a containerized application in a few simple steps.

Example scenario

Let's consider deploying a robust microservices application consisting of a React frontend, a Node.js backend, and a MongoDB database hosted separately via AWS DocumentDB. Our frontend provides the user interface, the backend handles business logic, and the database securely persists data.

Step 1: File structure

Defining the file structure is the foundational and most important part of this setup. It looks like this:

main.cpp C++

```
myapp/
├── frontend/
│   ├── Dockerfile
│   ├── app/
│   │   └── build/
│   └── copilot/
│       └── manifest.yml
├── backend/
│   ├── Dockerfile
│   ├── app/
│   │   ├── package.json
│   │   ├── api.js
│   └── copilot/
```

File structure

> **Note:** Before proceeding, ensure the copilot-cli is installed according to AWS Copilot installation guide.

Ask

Once the file structure is set, we can navigate to the root folder, in this case, the root folder is `/app`.

Step 2: Initialize application

We can initialize our Copilot application to create an organizational namespace using the following command:

main.sh Shell

```
copilot app init myapp
```

Initialize application

Step 3: Initialize environment

After initializing the application, the next step is to define isolated environments for secure and scalable deployment of our services. The command below shows how we can create and configure an isolated environment named `dev` to deploy our services:

Ask

main.sh Shell

```
copilot env init --name dev --profile default
```

Initialize environment

Step 4: Service initialization

Now that the environment is set up, we can initialize the frontend and backend services. Let's start by configuring and initializing the frontend service. This service is containerized using an NGINX server that hosts our React application's static files. The playground below contains the Docker file for our frontend application and the command to initialize the frontend using the Copilot CLI command:

Co...

```
cd frontend
copilot svc init --name frontend --svc-type "Load Balanced Web Servic
```

Ask

Initialize frontend

Following this, configure and initialize the backend API service, which uses Node.js to manage our business logic:

Co...

```
cd ../backend
copilot svc init --name backend --svc-type "Backend Service" --docker
```

Initialize backend

For database integration, we'll securely store our database credentials in an environment-specific secrets file. Create a file at `copilot/environments/dev/secrets.yml` containing the database details:

index.yaml YAML

```yaml
DB_ENDPOINT: <your-documentdb-endpoint>
DB_USERNAME: <your-db-username>
DB_PASSWORD: <your-db-password>
```

Ask

Environment variables

With these configurations in place, we can now deploy our entire environment.

Step 5: Deployment

We can deploy the services. We can use Copilot's environment deployment method by executing the following command:

main.sh Shell

```
copilot env deploy --name dev
```

Deploy environment

This command automatically provisions all infrastructure and deploys both frontend and backend services based on their defined manifests and Dockerfiles.

Ask

Step 6: Monitoring

To ensure the deployment succeeded and to monitor the application, run the following Copilot commands:

Che...

```
copilot svc status --name frontend --env dev
copilot svc logs --name frontend --env dev
```

Check logs frontend

Managing multiple environments

Once environments like `dev`, `test`, and `prod` are initialized using `copilot env init`, services can be deployed to each environment independently. This separation is more than organizational; each environment runs in its own isolated VPC with distinct subnets, security groups, and service discovery namespaces.

Behind the scenes, Copilot orchestrates ECS service deployments using AWS CodeDeploy. It supports deployment strategies such as:

- **Rolling updates:** It gradually replaces old versions with new ones.
- **Blue/green deployments:** It launches the new version alongside the one, shifts traffic, and enables quick rollback.

Ask

These strategies are implemented without manual configuration, reducing the likelihood of human error and increasing deployment resilience. Developers benefit from consistent infrastructure and safer deployments, especially when testing new changes across environments. Logs, metrics, and health checks are scoped per environment, simplifying debugging and monitoring.

Copilot also maintains consistency in image versioning and resource templates across environments, ensuring that what runs in production has passed through earlier stages unaltered. This model reinforces the principles of repeatable deployments and safe delivery pipelines.

CI/CD integration

AWS Copilot integrates seamlessly with AWS CodePipeline and CodeBuild for setting up CI/CD pipelines, enabling automated deployments with minimal manual intervention. This integration aligns naturally with Git-based workflows. Copilot manages end-to-end workflows from code commits to environment-specific deployments, streamlining continuous delivery processes. The illustration below shows CI/CD workflow and ECS integration with Copilot:

CI/CD workflow and ECS integration with Copilot

Ask

We can automate the deployment using Copilot's pipeline feature by executing the following command:

main.sh Shell-20

```
copilot pipeline init
```

Copilot interactively prompts us to select our Git repository provider, such as GitHub or AWS CodeCommit, specify the repository and branch to monitor, and determine which environments the application should be deployed to, for example, dev, test, or prod.

Upon completion, Copilot generates a detailed `pipeline.yml` file that explicitly defines the stages of the CI/CD pipeline. These stages include:

- **Source stage:** It defines the repository and branch monitored for code changes. Whenever a new code is pushed, this stage initiates the pipeline execution.
- **Build stage:** It utilizes AWS CodeBuild to compile and build the container images from the application source. This step automatically manages the build environment, dependencies, and runs specified tests to ensure quality.
- **Deploy stage:** It automates deployment to Copilot environments (e.g., dev, test, and prod) sequentially. It leverages Copilot-managed manifests to provision necessary resources, ensuring reliable and consistent deployments.

Ask

After configuring the pipeline, deploy it with the following command:

main.sh Shell

```
copilot pipeline deploy
```

This command instructs Copilot to provision all necessary AWS resources, such as CodePipeline, CodeBuild projects, IAM roles, permissions, and artifact storage, through CloudFormation. Copilot ensures that each stage is correctly configured, enabling secure and efficient pipeline executions.

By integrating Copilot and CodePipeline, we establish a robust CI/CD process that automates deployments, reduces manual overhead, enhances consistency, and accelerates the development lifecycle.

Promoting across environments using artifacts

One of Copilot's core strengths is its artifact-driven promotion system. After a service is deployed to an initial environment, such as `test`, the same container image (stored in Amazon ECR) can be reused for deployment to another environment like `prod`.

Ask

This artifact reuse ensures consistency and removes the need to rebuild code during promotions. Because the image is immutable and version-tagged, the

same version of the application can flow through multiple stages of the pipeline without change.

To control promotion flow, Copilot allows the configuration of manual approvals between pipeline stages. For example, a promotion to production can require explicit approval, while earlier stages may promote automatically.

By codifying pipeline stages in `pipeline.yml`, teams benefit from version-controlled, repeatable delivery processes that eliminate manual configuration drift and align with modern DevOps practices.

IAM and network configuration

Security and service-to-service communication are critical components of any production-grade deployment, and AWS Copilot integrates IAM and networking configurations to make these elements both secure and automatic. Every service deployed with Copilot requires precise access permissions to interact with other AWS services, as well as well-scoped networking rules to communicate internally or externally. Instead of leaving these configurations to chance or manual setup, Copilot provisions IAM roles and network infrastructure on our behalf and makes sure to follow AWS best practices for isolation, access control, and observability.

IAM roles and ECS tasks deployed via Copilot

When deploying applications with Copilot, IAM plays the role of gatekeeper. When deploying ECS tasks, Copilot automatically creates and attaches two primary IAM roles:

- **Task execution role:** It grants ECS permissions to pull container images and manage logging to CloudWatch.
- **Task role:** It allows the containerized application to access other AWS services (e.g., DynamoDB, S3).

Ask

For instance, a backend service interacting with DynamoDB and writing logs to CloudWatch receives only these precise permissions.

IAM association with ECS

The IAM roles assigned by Copilot are embedded directly within ECS task definitions, strictly adhering to the principle of least privilege. Developers can extend or further restrict permissions explicitly by modifying the `manifest.yml` file, where additional IAM policies can be defined. IAM roles also securely control access to application secrets stored in services like AWS Secrets Manager or Parameter Store, define logging capabilities, and restrict interactions with AWS APIs.

This automated, structured approach ensures services never receive excessive permissions, significantly reducing risks of unauthorized access or data disclosure, while retaining the flexibility required by evolving application requirements.

Secure networking and discovery across services

Copilot deploys each environment (e.g., dev, test, and prod) into its own isolated Amazon VPC. Within this VPC, it provisions private and public subnets, route tables, and internet/NAT gateways. Services that require public access, such as APIs, are routed through an Application Load Balancer attached to public

Ask

subnets, while internal microservices are hosted in private subnets with no direct exposure to the internet.

All services are registered in AWS Cloud Map, allowing internal communication via DNS-based discovery. This eliminates hardcoded IPs and enables seamless communication between services using their logical names.

Security groups are configured automatically to allow only necessary ingress and egress traffic, whether between services or from the load balancer. These rules are tuned based on the service's exposure level, protocol, and port configuration.

The result is a secure, maintainable network configuration where services can reliably communicate without security risks and manual intervention.

Logs and deployment resilience

Monitoring and resilience are core components of operating applications in production, and AWS Copilot incorporates both through integrated logging, deployment strategies, and rollback capabilities. These features enable teams to observe application behavior, trace failures, and ensure that deployments can be rolled back quickly and safely when issues arise.

Viewing logs

Copilot automatically configures Amazon CloudWatch Logs for every deployed service. Developers can stream logs directly from the command line using:

```
copilot svc logs --follow
```

This command retrieves real-time log output from the service's ECS tasks. It supports filtering by environment, service name, and time window, making it useful for debugging and verifying deployment success.

Ask

Because logs are automatically centralized in CloudWatch, developers don't need to configure log drivers or worry about where logs are stored. Logs can also be viewed in the AWS Console and integrated with CloudWatch Logs Insights for advanced querying and analysis.

Identifying rollout issues and misconfigurations

Copilot leverages AWS CodeDeploy to manage the deployment of services, which introduces advanced capabilities such as health checks, traffic shifting, and automated rollback. If a newly deployed version of a service fails health checks or fails to stabilize, CodeDeploy can automatically trigger a rollback to the last known good version.

The following command provides a snapshot of the deployment status, including current task health, running revisions, and load balancer target group states:

```
copilot svc status
```

This command is crucial for identifying stuck or failed deployments.

Common deployment issues Copilot helps surface include:

- ECS tasks stuck in a `PENDING` state due to resource constraints
- Health check failures causing tasks to be replaced repeatedly
- Incorrect environment variable or secret configurations

When combined with log output, these insights give developers a clear path from symptom to root cause, improving recovery time and reducing operational overhead.

Copilot also integrates seamlessly with AWS X-Ray for tracing requests and analyzing service dependencies, enhancing the ability to observe and troubleshoot production systems.

Ask

Conclusion

Ask